



Real-Time Interactive Music Score Tracking Assistant using Microphone Input

Student Name:	Kyle Hellström
Student ID:	22343261
Course:	BSc in Computer Systems (LM051)
Supervisor:	Annette McElligott
Academic Year:	2025/2026

Department of Computer Science and Information Systems
Faculty of Science and Engineering
University of Limerick

Submitted to the University of Limerick for the degree of BSc in Computer Systems
Academic Year 2025/26

Declaration

I herewith declare that I have produced this report without the prohibited assistance of third parties and without making use of aids other than those specified; notions taken over directly or indirectly from other sources have been identified as such. This report has not previously been presented in identical or similar form to any other Irish or foreign examination board.

This work was produced under the supervision of Ms. Annette McElligott at the University of Limerick.

Limerick, 2026

Table of Contents

Declaration.....	i
Table of Contents.....	ii
1 Introduction and Objectives.....	1
1.1 Learning Music.....	1
1.2 Interactive Music Assistance Apps.....	1
1.3 Project Objectives.....	2
1.4 Project Approach.....	2
1.5 Report Structure.....	2
2 Research.....	3
2.1 Introduction.....	3
2.2 Interactive Music Assistance Apps.....	3
2.2.1 Lesson-based Apps.....	3
2.2.2 Notation and Score-based Apps.....	3
2.2.3 Limitations.....	4
2.3 Music Theory.....	4
2.4 Audio Analysis.....	5
2.5 Image Conversion.....	5
2.6 Audio-to-Score Alignment.....	5
2.7 Key Findings.....	6
2.8 Summary.....	6
3 Methodology.....	7
3.1 Introduction.....	7
3.2 Architecture.....	7
3.3 Design.....	8
3.3.1 Audio Processing.....	8
3.3.2 Score Processing.....	8
3.3.3 Pitch Detection.....	9
3.4 Evaluation.....	9
3.5 Summary.....	10
4 Development.....	11
4.1 Audio Input.....	11
4.2 Score Display.....	11
4.3 Audio Analysis.....	12
4.4 Architecture.....	13
4.5 Monophonic Detection.....	13
4.6 Polyphonic Detection.....	14
4.7 Detection Accuracy.....	15
4.8 Dilemmas.....	16
5 Conclusions.....	18
5.1 Introduction.....	18
5.2 Challenges.....	18
5.3 Development.....	18
5.4 Summary.....	18
References.....	19
Appendix A - Pitch Detection Testing Dataset.....	22

1 Introduction and Objectives

1.1 Learning Music

When choosing to learn how to play a musical instrument, an individual can select from several teaching methods. They can take private lessons with the assistance of a teacher which affords the individual personalised guidance. Alternatively, the learner can decide to take lessons as a member of a group, where collaboration teaches harmony with other learners, while enhancing a person's active listening skills. An individual also has the option to take lessons online, with or without a tutor, which provides the individual the opportunity to learn at their own pace where they may avail of a myriad of resources. Apart from these methods, an individual can be self-taught, where they might choose to learn by ear, and they may also engage with these resources. In their musical educational journeys, the majority of these persons will learn how to read sheet music, whereby they will encounter several of the key components of music theory (cf. Section 2.3), pitch, notes, chords, tempo, notation, timbre, dynamics, and articulation.

I have been playing music for just over a decade. I was taught by two teachers, each with a different style of teaching. My first teacher was a classical concert musician who taught me how to play the piano for concert performances while the second teacher taught me the music theory and techniques that I lacked. Between lessons, I found the lack of immediate feedback from a tutor problematic. It occurred to me that I would benefit from a tool that would provide ample feedback based on my performance of live audio, during practice using my own sheet music. Even though I failed to locate such a tool (cf. Section 1.2), I decided to use the opportunity of my Final Year Project to research the feasibility of designing and developing such a tool, thereby combining my knowledge of music with my software development skills.

1.2 Interactive Music Assistance Apps

The online resources that assist individuals in their learning of music offer features that support practice and reading musical notation. These apps can be categorised based on whether they are focused on providing resources and teaching an instrument, or converting and reading musical notation.

Applications like Flowkey, Yousician, Fender Play, Synthesia, TrueFire, and the Simply company's products provide a large library of curated songs, accurate audio input from instruments either physically or through a microphone, and feedback on an individual's performance (Flowkey GmbH, 2026; Yousician Ltd, 2026; Simply Ltd, 2026; Synthesia LLC, 2026; TrueFire Inc, 2026). These apps all excel in creating engaging lessons that return feedback to individuals based on mainstream songs. However, they lack customisation in media that is not manually uploaded and curated for the software causing an eventual limitation of the songs that are able to be taught. The types of content in many of these apps primarily focuses on teaching beginner and intermediate musicians appropriate songs based on their musical abilities (Fender Musical Instruments Corporation, 2026) which restricts the complexity of the songs that are able to be taught. Synthesia differs in that the song library contains sample songs and supports the upload of custom songs, however, the caveat is the requirement of a connection from a digital piano to the software to accept direct instrument input instead of using live audio (Synthesia LLC, 2026). The products created by Simply offer instructional lessons for a diverse range of instruments and skills, where they offer singing, guitar, piano, and piano XR (Simply Ltd, 2026) which implements extended reality into learning where feedback is given through virtual assistance.

Converting and reading sheet music online can be achieved through the apps Musescore (Muse Group, 2026), PlayScore 2, and ScanScore. These accept the upload of sheet music, where Musescore requires specific file formats while the other apps convert the uploaded images into a digital format to allow for playback of the uploaded score. An app with similar functionalities is Chordify (Chordify BV, 2026) which

provides audio to chord conversion and displays the resulting tablature for practice. Software like Ultimate Guitar Tabs (Ultimate Guitar, 2026) provides expansive libraries of professionally and user-made chords and tabs for practice for guitar, piano, and ukulele. These apps are ideal for converting and viewing music notation in a digital format but are not designed for the return of feedback as they lack the input of audio and the alignment of audio-to-score which is an essential requirement for the proposed project.

Most of the functionalities required for this project exist separately in publicly available software, yet fail to create an application that contains the combination of these features into one expansive music assisting and learning tool.

1.3 Project Objectives

The objective of this work is to research and develop a web application that allows an individual to upload sheet music and receive feedback based on live audio performance. This will be achieved as follows:

1. Review of the functionality of popular music learning apps will be conducted to understand the feasibility of the project and the technologies required to create an operational prototype of the proposed application (cf. Section 1.2).
2. Summary of the aspects of music theory relevant to this work (cf. Section 2.3).
3. Presentation of findings from research articles on the processing of live audio for transcription (cf. Section 2.4), manipulation of a scanned music sheet (cf. Section 2.5), the alignment of audio to the score (cf. Section 2.6), and key findings found (cf. Section 2.7).
4. Design, development, and evaluation of the proposed system (cf. Section 3).
5. Description of development, steps taken, and challenges faced (cf. Section 4).

1.4 Project Approach

The project will research relevant music theory concepts, audio processing and digital transcription, music score conversion, and audio-to-score alignment to identify the technologies required for development. The evaluation of existing interactive lesson apps will influence the design of the proposed system allowing the development to focus on the technology required instead of the user interface and experience. The development of the project will follow a modular workflow, separating score conversion from the audio processing, where they can be integrated to focus on the alignment aspect. This structured approach allows for the isolated testing and evaluation of components during development before the final implementation. The project will be a browser-based application using the software languages of Hypertext Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript. The functionalities consist of converting sheet music into a digital format, transcribing live audio after pre-processing into a symbolic format to allow for the comparison of the audio to the score. This comparison will be guided by the audio-to-score alignment where real-time feedback will be prioritised.

1.5 Report Structure

This chapter presented the rationale for the proposed tool, a summary of what the project sets out to achieve and how this work will be accomplished. Chapter 2 will present the aspects of music theory relevant to this work, the findings from literature on the various techniques that could be utilised to manipulate a scanned music sheet and process live audio input for comparison. Chapter 3 will detail the design of the proposed solution. Chapter 4 will document the developmental progress and challenges faced, while Chapter 5 will present conclusions and make recommendations for further work.

2 Research

2.1 Introduction

During the process of identifying the scope of this project, a review of the tools, technologies, and research literature was conducted to determine whether solutions to the project already exist. A critique of the existing solutions, an evaluation of their strengths and limitations, and identification of the gap in the market that this project aims to fill will be included (cf. Section 2.2). Having enumerated the objectives of the proposed project (cf. Section 1.3), research was conducted on the music theory that is required for learning music and signal processing (cf. Section 2.3), techniques to process live audio for feature extraction (cf. Section 2.4), converting images into a digital format (cf. Section 2.5), and an analysis of methods to align the processed audio to the converted score (cf. Section 2.6). Section 2.7 presents the relevance of these research findings on the proposed tool.

2.2 Interactive Music Assistance Apps

The applications mentioned in Section 1.2 outlined the functionalities available in existing software related to the aim of the project and the areas in which they are lacking. These technologies were accumulated through targeted searches of product websites, searching through genres on app distributors for mobile, and investigations of apps that assist individuals learn and practice music online. The resulting apps introduced the key features that would need to be taken into consideration while developing this implementation.

2.2.1 Lesson-based Apps

The apps focused on lessons and providing resources aiming to teach music through predefined lessons of popular songs. The song libraries that are provided are expansive yet limiting as the lessons are focused mainly on beginner and intermediate players. This is due to either the requirements that the software is based on where intricate songs are not prioritised, or a technological limitation in pitch detection where complicated songs may overlook notes during performances due to the dissection of complex frequencies. This limitation allows for a gentle progression while learning as the difficulty increases based on performance. This is especially the case for those who partake in lessons with an instructor and learn sheet music that is not present in these apps, or that have been modified from the original piece. The feedback provided by these apps detects note correctness, timing accuracy, and some rhythm errors which assist the users as they play. This is provided through the practice and play modes where options for tempo, metronome, automatic playback of the song, looping for certain bars and measures, and automatic tempo to speed up the alignment as a user plays. These features are ideal for lesson apps as they provide a guide as a song is taught.

The audio input methods provided by these applications are mostly audio input and direct Musical Instrument Digital Interface (MIDI) input however, applications like TrueFire accept direct input from an instrument through an audio interface (TrueFire Inc, 2026). Accepting live audio allows for countless users to use these applications and learn from the predefined collection of songs and lessons.

2.2.2 Notation and Score-based Apps

The apps related to music notation and score conversion allow for import and export of scores where MuseScore requires the file format MusicXML (.mxl or .musicxml), or MIDI (Muse group, 2026) to visualise scores in a digital format in their software, while ScoreScan and PlayScore 2 use an implementation of optical music recognition (OMR) (Organuma Ltd, 2026; ScanScore Ltd, 2026a) to process an image from a photo format or Portable Document Format (PDF) into a digital format.

2.2.3 Limitations

These lesson-based apps neglect the users who require feedback for advanced songs that may not be in their libraries and lack the functionalities for user-supplied scores and tabs. The feedback handled by these apps is their prevailing strength where they falter in the customisation of their song libraries. The notation and score-based apps contain the contrasting limitation where they provide a method of converting images or varying formats but lack the provision of feedback as the user performs their uploaded score. Additional information regarding the techniques and technologies used in these apps are limited and further research has been completed based on audio analysis, processing, and topics surrounding OMR and additional methods of image conversion in Sections 2.4 and 2.5.

2.3 Music Theory

Learning music primarily revolves around the theory behind the interaction of sound with the environment. This section will outline the theory required to address the challenges surrounding signal processing and polyphonic analysis. This theory revolves around the frequencies that we hear, and the oscillation speed of these frequencies determines the pitch that is produced. The unit for pitch is defined as hertz. Frequencies and pitches are the underpinning requirements of pitch detection algorithms. A note is a pitch and the duration it is played, where the middle A note of a piano has 440Hz, and the duration would be specified through the use of musical notation and symbols.

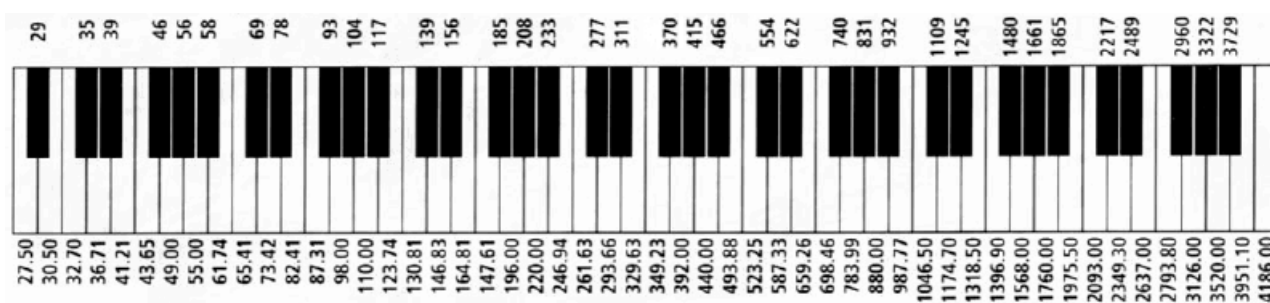


Figure 1: Piano key frequencies (Sengpiel, 2014)

Tone refers to the pitch, duration, and quality of a sound that is produced. This quality determines the sound an instrument makes compared to other instruments, similar to how accents differ. In music, this is referred to as timbre. The subtle frequencies encapsulated within a note are called overtones. These overtones can be separated into harmonic and inharmonic overtones. The lowest yet loudest frequency produced by a note is called the fundamental in the overtone series and the first harmonic in the harmonic series. The harmonic series consists of integer products of the fundamental frequency where the second harmonic would produce a note twice as high in frequency as the first harmonic, and would produce an octave when played together on an instrument. This can be seen in Figure 1 where the notes of Hertz 220 and 440 both align with the note 'A'. The third harmonic returns a perfect fifth, which roughly equates to the note 'E' with the actual hertz of 659.26 where the inaccuracy is due to the method of tuning instruments. The overtones that are not present in the harmonic series are the inharmonic overtones and largely contribute to the tone and timbre of the sound. This is due to the differing volumes of each harmonic depending on the instrument. In traditional Western music, instruments are tuned based on an equal temperament from 440Hz where each note is divided into twelve equal steps for the twelve notes present within an octave in most instruments.

A chord is the combination of notes played simultaneously to produce harmonies. These harmonies can result in consonance or dissonance depending on the context and culture. A single note produces its fundamental frequency and a myriad of overtones and when combined with additional notes, the resulting frequency becomes challenging to disassemble to determine the fundamental frequencies of each note. Monophony is a singular melody from an instrument where the song progresses using a single note. Once harmony and chords are added, this becomes polyphony as there are multiple melodies being played

simultaneously. Isolating the fundamental frequencies in polyphonic audio becomes increasingly difficult as the harmonic and inharmonic overtones intensify. The methods in which these notes are played are determined by tempo, dynamics, and articulation. Tempo defines the speed of the piece to be played which allows for mellow or energetic performances. The dynamics of a song determine how loud or soft notes should be played to add emotion to a piece, along with articulation describing the speed at which each note is to be played. These features add musical expression to a piece of music, however, they may cause performances to stray from the precise notation written in scores while practising.

2.4 Audio Analysis

Audio is usually captured by a microphone, which takes the frequencies in the environment, and through the use of a delicate diaphragm, digitally converts the audio signal into a digital signal. The process of extracting features from an audio signal often depends on time-frequency representation of the data where a common implementation is through the use of a spectrogram (Kurth et al., 2007). This spectrogram is further processed to differentiate the twelve notes on a piano, where each octave within the spectrogram is converted into a single vector containing each note, also known as the pitch class. The values of each pitch class for a given timeframe represent the probability of the notes actively being produced. Fourier Transform is used to transform an audio signal into a representation of the strength of various frequencies contained within the original input signal. This helps determine the fundamental frequencies within an audio signal and the basis of feature extraction. Short-Time Fourier Transform (STFT) is a method of converting audio into data with the correct format for chromagrams by applying a Fourier Transform on the audio in non-zero time frames (Fremerey et al., 2009), which is ideal for processing live audio. These chromagram representations produce functional results for monophonic detection, however, these lack in robustness when compared to polyphonic detection due to noise in the signal interfering with the feature extraction (Kurth et al., 2007; Fremerey et al., 2009).

2.5 Image Conversion

The format used for representing musical notation is primarily MusicXML where alternatives are NoteWorthy Composer (NWC) and Notation Interchange File Format (NIFF). These formats are occasionally used to display sheet music, but the leading format, which is used in Musescore, PlayScore 2, ScanScore, and for OMR conversion, is MusicXML (Organum Ltd, 2026b; ScanScore Ltd, 2026b). This format is used in the software Audiveris, which is an open-source OMR tool designed to convert scanned music scores into MusicXML for exporting (Audiveris, 2026). The licence for Audiveris is a GNU Affero General Public License version three which allows for the modification of the software as long as it is redistributed under the same license and availability of the modified source code. Alternative software for OMR is software such as OpenOMR (Greenjava, 2014), which implements similar features but lacks in development and improvements that Audiveris has been supported with as the GitHub repository (Audiveris, 2026) has been continuously developed compared to the eleven-year stagnation OpenOMR has endured.

2.6 Audio-to-Score Alignment

The alignment of the audio to the score can be achieved through the use of a Dynamic Time Warping (DTW) algorithm (Kurth, 2007). DTW is an algorithm designed to align two sequences based on non-linear tempo variability, where the sequences would be the chroma vectors of the audio and the symbolic transformation of the converted sheet music. A cross-similarity matrix is created from these two vectors to compare the distance between each value. This finds the optimal path across the matrix to align the audio vector to the score vector.

2.7 Key Findings

This research shows the existence of the features of music sheet conversion using OMR, audio processing for feature extraction, and alignment using DTW across current solutions, yet they remain independent from each other. The combination of these features would allow the feedback from music lesson applications to be provided to any sheet music an individual owns. This is the opportunity that this project will work upon and develop.

2.8 Summary

Through a thorough review of literature and existing technologies, the key concepts supporting the proposed system have been identified. Applications focused on lessons and resource provision were analysed to discover their functionalities, distinguishing their strengths and limitations. Relevant music theory essential for signal and music score processing were also discussed to highlight the challenges related to pitch detection and the various techniques that are required for their processing. The research demonstrates how the features of converting a score to a digital format, processing live audio, and the alignment of these features exist independently, while their integration remains limited.

3 Methodology

3.1 Introduction

Having completed a review of the literature and technologies that separately implement the features required for the proposed project, an elaborate explanation of the decisions that will be made during the project's development will be provided in this chapter. An overview will be provided on the architecture of the system (cf. Section 3.2), the design and steps required of the system for each of the required elements (cf. Section 3.3), and the methods of evaluation that will be used (cf. Section 3.4).

3.2 Architecture

The development of the project will progress using a staged approach inspired by the waterfall model for the developmental life cycle of the project (cf. Figure 2). This approach adheres to the phases of planning requirements, designing, developing, testing, deployment, and maintenance. The requirements and design will be established in this chapter along with the developmental and testing plans. The deployment of a feature will consist of uploading the latest files to a release branch in the source control employed, specifically GitHub. Maintenance will be accomplished as further testing and integration of additional code are implemented.

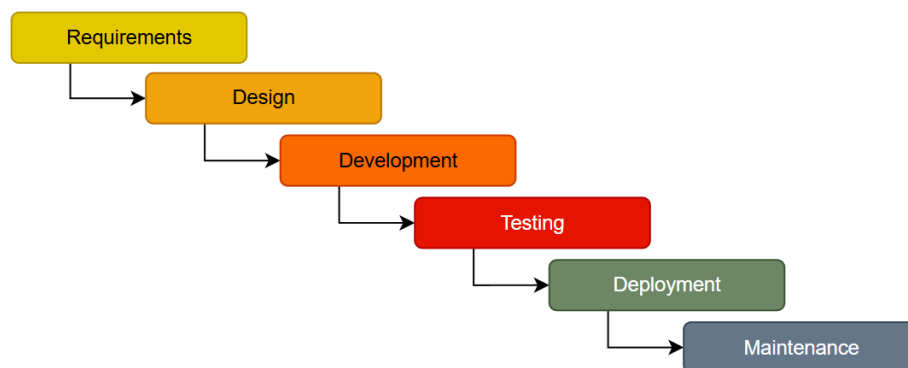


Figure 2: Visual representation of the waterfall model

The architectural pattern chosen for the development will be Model-View-Controller (MVC) as the separation of the user interface from the representation of the required information through the use of an intermediary class which creates a logical pipeline of data that will ensure the organisation and structure of the code. MVC has been chosen over other patterns such as Model-View-Presenter (MVP) and Model-View-ViewModel (MVVM) due to various factors. The separation of the view from the model using controllers will aid in the creation of tests due to the pre-configured classification of data where MVP additionally prioritises the presentation of the view and MVVM focuses on the binding of data between the view and the viewmodel which is not a priority for this project. Furthermore, the experience I have acquired over the duration of developing production-level code during university and throughout work placement encourages the use of MVC.

To create this architecture, I will create three folders for the model, view and the controller. These folders will separate the view, containing the HTML, CSS, and JavaScript logic files organising data for the User Interface (UI), from the model, consisting of the representation of the required data for processing and visualisation, and the controller, which will

```
ProjectRoot/  
├── Model/  
│   └── values.js  
├── View/  
│   ├── index.html  
│   ├── index.css  
│   └── UI.js  
└── Controller/  
    ├── audioController.js  
    └── scoreController.js
```

Figure 3: Prototype MVC file structure

contain files to process the different aspects of the system for comparison. Figure 3 shows the file structure that will be used and elaborated on during development.

3.3 Design

The system will be developed with a focus on following a sequence of steps to provide the comparison of a music score to live audio. This will cover the processing of sheet music, extraction of data from an audio signal, the formats required for the input of both the audio and the score, and the alignment of the gathered information.

3.3.1 Audio Processing

The WebAudio API (W3C, 2024) will allow for the interaction of an individual's microphone through a browser and will contain methods used in the processing of audio (cf. Section 2.4). As the input of audio will come from an individual's microphone using the WebAudio API, the audio format will depend on the sound card within the hardware of their device or the hardware of an external microphone. The channels of audio can range from one to more than thirty-two where the number of channels the project will work around is two. The bit depth and sample rate will be a challenging variable as technology can vary these ranges depending on a microphone's hardware where similar to the channel count, development and evaluation will use 16-bit audio with a sample rate of 44.1 kHz to 48 kHz. This means hardware worse than these values may be insufficient for testing.

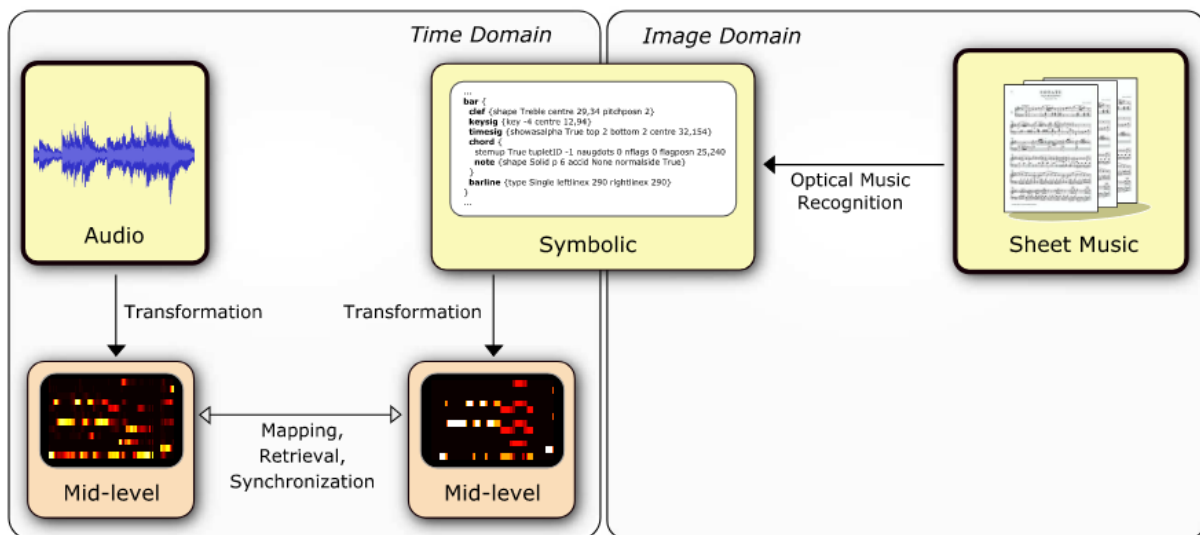


Figure 4: Illustration of data types and transformations required to perform cross-domain comparison between audio and sheet music (Fremerey et al., 2009)

A chromagram will be created from the audio through the use of the Meyda library (Meyda, 2026). As shown in Figure 4, this will be used for comparison to the score which will allow the ability of pitch detection from the processed score and audio.

3.3.2 Score Processing

Image processing will be achieved by the implementation of the Audiveris library as mentioned in Section 2.5 where the resulting MusicXML file will be parsed into a chromagram using OpenSheetMusicDisplay (OSMD). The computation of the OMR system will be attempted to be computed locally within the browser to allow users to import sheet music into their system considering the correct file format for conversion. The system will log the progress of the conversion to allow the user to observe whether it succeeds or fails in the browser's console. The success will depend on the accuracy of the technologies to be implemented as perfect conversion will depend on the condition of the score quality and the abilities of the OMR system

to convert the file. If the conversion cannot be executed in the browser alone then a back-end server will be required to handle the responsibility.

The application will progress in a similar manner to the aforementioned Figure 4 where the browser will accept the imported music sheet in the format of PNG, JPG, and PDF, which will be transcribed into MusicXML and displayed in the UI using .mxl and .musicxml file formats. MusicXML was chosen as it can be displayed with the implementation of the OSMD library (OpenSheetMusicDisplay, 2026). For the input format into the browser, OMR will be redundant for the MusicXML format as this is the symbolic format required for the audio-score comparison as mentioned in Section 2.4.

Legal constraints are present with the storage of data since sheet music may not be permitted to be stored or publicly shared. The creation of the server will prioritise local persistence as externally hosted servers will introduce the complexity of complying with General Data Protection Regulation (GDPR) due to the possible handling of licensed scores. The current alignment technologies show how alignment can be done (cf. Section 2.6), however the use of convolutional neural networks presents a viable opportunity to increase the speed of alignment and data conversions but lacks training data (Hawthorne, 2018) which is why this project will not implement machine learning.

3.3.3 Pitch Detection

The alignment described in Section 2.7 will be implemented using the package dynamic-time-warping via the npm package manager (dynamic-time-warping, 2016). This alignment will allow for the use of monophonic and polyphonic pitch detection when comparing the audio to the score. The DTW function will require the computation of the distance between the two chromagrams which will require testing with the formulas contained in the Math.NET library (Math.Net Numerics, 2024) as the optimal formula has not been specified. The comparison will be completed based on the pseudocode shown in Figure 5.

Feedback will be largely inspired by the features shown in the lesson-based apps as the functionalities that are presented in these apps consist of hearing a playback of the song, the ability to play at an individual's personalised pace and observe the difference in detected pitch from the audio compared to expected pitch from the score through the use of instruments with reasonable overtones. Similarly, the UI will be largely inspired by these applications, although not prioritised as the implementation of the aforementioned features are the focus of this project rather than the aesthetics.

Pitch Detection Pseudocode:

Input:

audioChroma
scoreChroma

If audioChroma is null OR
scoreChroma is null
return false

Create DTW model using:
audioChroma,
scoreChroma,
distance function

Calculate DTW distance between the
two chromagrams

Normalise distance value

If normalised distance is below
threshold
return true
Else
return false

Figure 5: Pseudocode for chroma value comparison

3.4 Evaluation

Once development progresses, the evaluation will use a piano, guitar, and vocalisation for ensuring accuracy from public datasets and reliability through the repeated testing of each feature. The processed audio will endure vigorous testing in varying environments using manual microphone input and piano input data from the Aligned Classical Piano Audio and Score (ACPAS) dataset (Cheriell, 2021), OMR will undergo the execution of sheet music acquired from datasets, such as International Music Score Library Project (IMSLP) which is a digital library containing public domain PDFs of music scores (International Music Score Library

Project, 2026), MusicXML's website where they source sites containing publicly available music scores (Makemusic, 2022), and manually sourced score. The alignment will be tested using the mentioned instruments alongside the datasets of scores. The results will be stored in log files and in the code where unit tests will ensure the consistency of the features as development progresses. Success will be measured based on the accuracy of the pitch detection for monophonic and polyphonic signals, the reliability of alignment of the audio to the score based on values returned by DTW functions, and overall usable feedback provided in the final product.

3.5 Summary

This chapter clarifies the architectural choices that will be made during the development of the project. An outline of the libraries to be used for the image conversion, audio processing, and alignment of these features have been provided based on the literature reviewed in Section 2, along with the specifics of formats of the audio and score inputs, the constraints that have restricted the development, and the methods of evaluation that will dictate the direction of development.

4 Development

The development of this system has been documented on GitHub (Hellstrom, K., 2025a) where the code referenced within this chapter is contained within this source control. References to most of the methods and concepts used are documented at the bottom of the files where they are implemented.

4.1 Audio Input

The development began with creating a simple HTML and CSS webpage with JavaScript to display the title of the project and create a simple layout for the required elements. The WebAudio API is used to take in a microphone signal and visualise it using graphs inside of HTML canvases for a time domain and a frequency domain. The time domain is used to display amplitude over time using a buffer which consists of segments of the amplitude in an array whose size is based on the Fast Fourier Transform (FFT) size from JavaScript's AudioContext analyser node which is initialised to allow for the visualisation and processing of an audio signal's time and frequency data. The domain graphs were visualised using the analyser node however, I encountered an issue with the WebAudio API's `getFrequencyByteData()` method as the size of the frequency bins, which were calculated as the AudioContext's sample rate divided by the analyser node's FFT size, were restricted based on FFT size, sample size and buffer size, limiting the detectable frequencies. This could be seen through the comparison of the processed estimated frequency to the frequency of guitar strings which were tuned using an online tuner on the application Ultimate Guitar Tabs and a mobile spectrogram called VocalPitchMonitor (TadaoYamaoka, 2025) displaying the hertz of a signal's fundamental frequency. This resulted in the observation of a difference of approximately 4 Hz which was due to frequency bins were not including all possible frequencies which forces the calculated frequency to map itself to the closest bin causing this discrepancy.

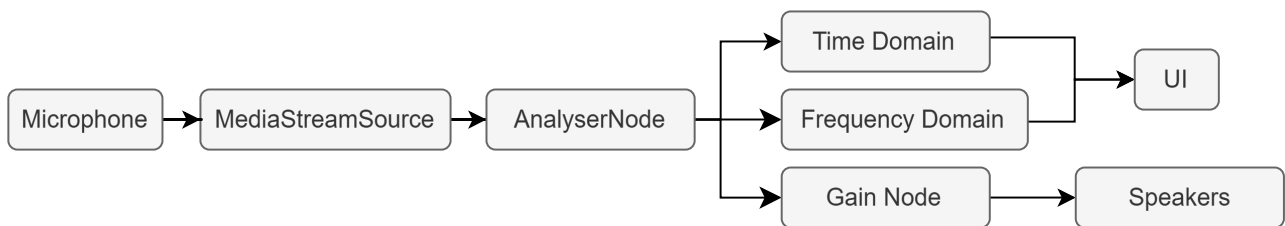


Figure 6: Pipeline of Initial implementation of Audio input

To ensure that the quality of the audio was as expected, I implemented the ability to listen to the audio input as shown in the pipeline displayed in Figure 6. This required the addition of audio gain to the system from the analyser node. It is initially set to zero allowing no audio to be heard, and is able to be toggled in the UI to set the gain to one which allows the audio to be heard.

4.2 Score Display

The system uses OSMD to display the music score and implements the methods within its library to create a cursor to access score data for each note along the score. To acquire the scores before having OMR functionality, public domain scores were required and was attempted to be obtained from IMSLP and ACPAS databases as described in Section 3.3. However, from my experience, these scores were mostly in the format of PDFs and had mediocre quality as they were antique scores. I managed to acquire an adequate number of music score images and MIDI files from the forenamed sources (cf. Section 3.3) and contain them within a folder labelled "Input" within the GitHub (Hellstrom, K., 2026b). Additionally, I found a source of public domain MusicXML files from a GitHub repository by manhtai (2024) which has been included in the GitHub and used for testing.

The conversion of the score could not be completed with only Audiveris as OMR only converts images into MusicXML where this does not include MIDI files. To convert MIDI files into the MusicXML format, it

requires an additional technology. The method I chose was through the use of Musescore as the alternative implementations, such as Music21 (Music21, 2025), Noteflight (Noteflight, 2026), Melobytes (Melobytes, 2026), would not execute within the browser. JavaScript’s package manager, npm, contains a package called webmscore (LibreScore, 2023) that I attempted to use to convert MIDI files to MusicXML to display them in OSMD however, this requires the use of WebAssembly or a back-end server. Audiveris returned similar issues where the browser alone was not enough for conversion as a Command Line Interface (CLI) was required to run batch code. This led to the creation of a container in Docker that was capable of these tasks as I had previous experience with Docker and was comfortable in creating the required Linux commands to install both Audiveris and Musescore onto an Ubuntu system to run on a local server to adhere to the constraints of the server described in Section 3.3.

I started creating my system using a Visual Studio Code extension called Live Server (Ritwick Dey, 2026) as it allowed me to live refresh my software during development. This was used until the implementation of the Docker container where the extension would not allow Representational State Transfer (REST) GET commands to be run which led to the creation of a node server on my local machine that connects to the container. As shown in Figure 7, this allows the system to send HTTP requests to the server for the file conversion and once the conversion is completed, returns the file to the browser and renders it using OSMD.

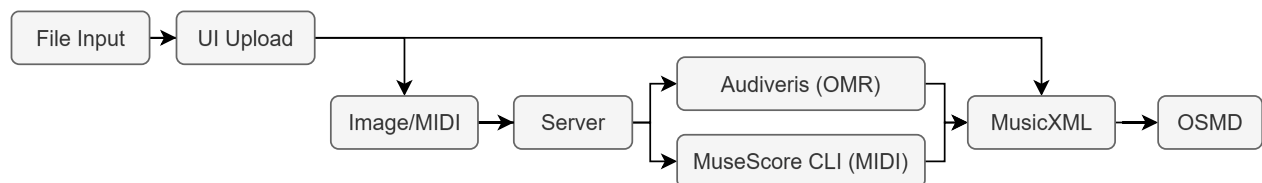


Figure 7: Pipeline of the Back-end Server Score Processing

Without this server, the conversion of music score files into MusicXML is not possible. However, MusicXML files that do not require conversion and are correctly formatted are still able to be rendered with OSMD, as they do not require server processing and OSMD runs stand-alone in the browser.

4.3 Audio Analysis

Through the implementation of the feature to listen to the browser’s real-time microphone audio, the signal was shown to be inconsistent and lacked the smooth signal required for accurate pitch detection as it appeared to oscillate due to a personally undiscoverable fault in the code related to the audio input. An example of this can be seen within the branch “Early-Gain-Implementation” in the forenamed GitHub repository for this system. This was further emphasised with a console deprecation warning from Meyda suggesting users to implement the AudioWorkletNode instead of the method called ScriptProcessorNode(). This led me to implement the AudioWorklet interface alongside the WebAudio API to separate the audio from the UI thread which results in lower latency and assists in real-time analysis without a large window for buffering. This change is represented in Figure 8 which displays the progression from microphone input to updating the UI data.

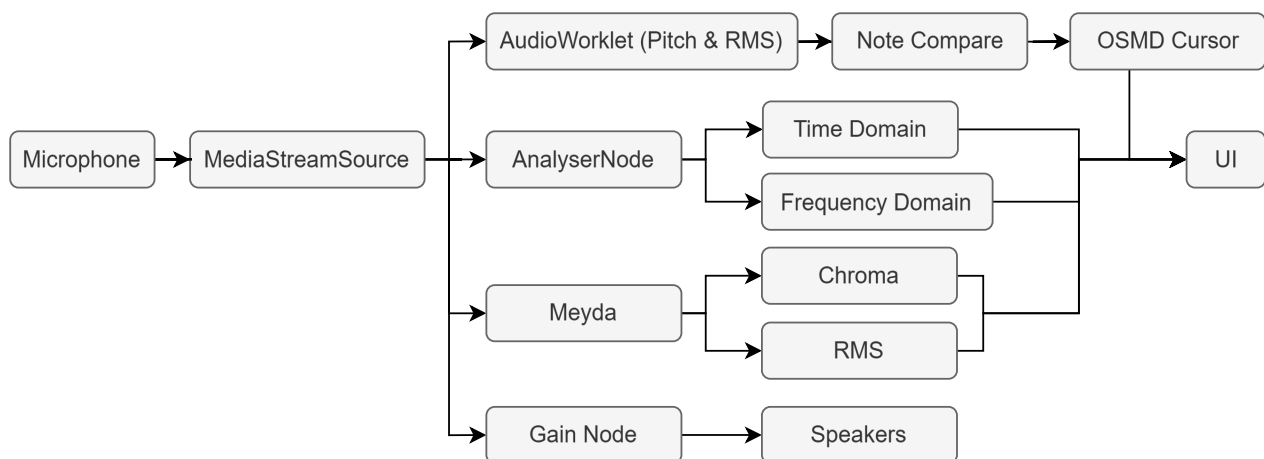


Figure 8: Pipeline of Initial implementation of monophonic audio input

The limitations of the WebAudio API influenced the decision to import Meyda as it accepts the signal provided by the WebAudio API and outputs numerous audio signal features which, upon further development, introduce rough monophonic detection. The features introduced include chroma values which represent pitch classes and are required for polyphony (cf. Section 2.4), and Root Mean Square (RMS). This RMS represents the average amplitude of the signal and indicates how strong the signal can be without distortion or whether the signal is silent which are values around 0.01. The integration of the Meyda package allowed for improved audio signal processing and output of more precise signal data as the detected frequency does not rely on frequency bins. However, the actual hertz of the fundamental frequencies were not able to get captured with Meyda alone. Precise frequencies, as seen in the paper by Pan et al. (2015) reveal how autocorrelation can be used to process the raw audio signal and produce data such as RMS and fundamental frequencies. The code used to implement this was sourced from a GitHub project ‘PitchDetect’ by Chris Wilson (Wilson, 2015) who includes a method whose parameters are a buffer, the same buffer as the time domain specified in Section 4.1, and the AudioContext’s sample rate. This data allows for the production of a fundamental frequency that manages to detect monophonic melodies but is slightly varied compared to the data processed by Meyda and represented through the chroma values due to differing processing calculations.

4.4 Architecture

The system initially was developed as the implemented features progressed spontaneously without strictly following any architecture. This required resolving with the application of audio-to-score comparison where gathering the acquired data into a single file became challenging. Adhering to the architecture specified in Section 3.2, an MVC pattern was implemented, separating the UI aspects of the code from the logic by creating controllers for the audio and score, a UI file for regulating where the graphs, signal data, and score control are managed, and a main file for initialising OSMD score, the Meyda chroma containers, and the browser’s window control. There is a singular model, labelled values.js, used for the audio and score in one constant variable in a values file that contains the data required for monophonic and polyphonic comparison to a score.

4.5 Monophonic Detection

When acquiring the notes used for comparison against the processed audio that are displayed on the MusicXML using OSMD, I attempted to gather the note information directly from the score’s file. To gather this information, a parser script for the Extensible Markup Language (XML) data from the score would be required to detect note, pitch, and additional relevant informational tags from the source code. This required a greater understanding of the creation of MusicXML files and appeared unnecessarily complicated for the task at hand. I delved further into the OSMD library documents, specifically the cursor functionality,

and happened upon the methods that provide information based on the location of the cursor which includes the note data. By gathering this data for each note along the music score, comparing it to the audio, and progressing the cursor location to the next note once a match is found (cf. Figure 9), monophonic pitch detection is achieved.

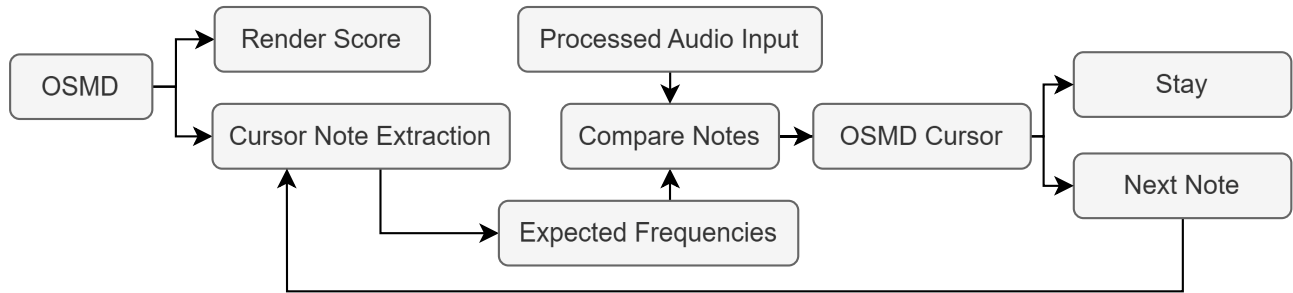


Figure 9: Pipeline of the monophonic score-to-audio processing

This system works efficiently for frequencies with minimal overtones and simple timbre instruments which can be seen with vocalisation, guitar, and piano. Although, notes below the third octave, which equates to frequencies below 130.813 Hz, which cause the detection to become delayed as the fundamental frequencies of lower notes are weaker than higher notes as the harmonics are more defined as seen in Figure 10. This is one of the major issues with creating polyphonic detection.

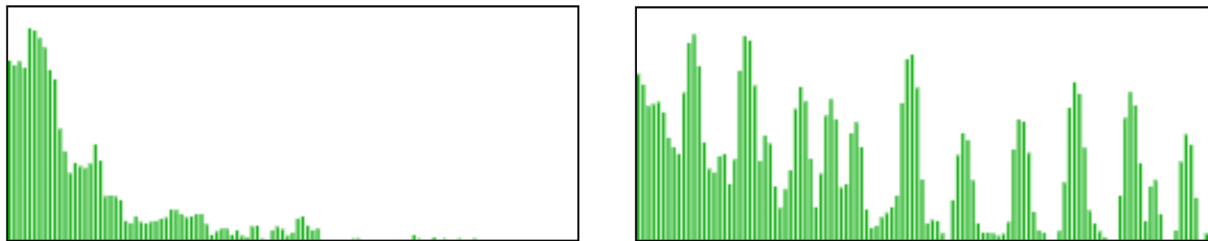


Figure 10: D3 (left) and F5 (right) played on a guitar

4.6 Polyphonic Detection

The first implementation attempted for polyphonic detection is inspired by the paper by Fremerey et al (2009) which implements the use of chromagrams and the comparison between the score and audio processing. Chroma values have been acquired and displayed in the system by using Meyda (cf. Section 4.3) yet the chroma values from the score required to create the chromagram has caused a halt in developmental progression. A solution was found by gathering the expected notes from the OSMD cursor and converting the data to chroma values using the method shown in Figure 11. The resulting values are then repeated multiple times in a new array. This repetition helps ensure that a detected note is actually being played instead of being a false positive, since it must match the correct value several times continuously.

```

export function freqToPitchClass(freq) {
  return ((Math.round(freqToMidi(freq)) % 12) + 12) % 12;
}

```

Figure 11: Conversion of frequency to pitch class code

This comparison is handled by the function displayed in Figure 12. It takes the chroma values from the audio and the expected frequencies from the score, ensuring they are both valid values and are populated. The score frequencies are then mapped into the repeated chroma array and then passed, along with the audio chroma array, into a new DTW instance. This calculates the distance between the two arrays based on a

chosen distance formula (cf. Section 3.3.3). The resulting distance is normalised, and if the result falls within a threshold, typically around 0.2, the system determines whether it is a match and returns a Boolean value indicating the result. This is executed in the audio worklet thread, allowing the comparison to be calculated in real-time and allowing the cursor to advance immediately.

```
export function compareNotes() {
  const detectedChromaHistory = values.audio.chromaValues;
  const expectedChroma = values.score.chroma;
  if (!expectedChroma || expectedChroma.length === 0) {
    return false;
  }

  if (detectedChromaHistory.length === 0) {
    return false;
  }
  const expectedChromaHistory = detectedChromaHistory.map(() =>
[...expectedChroma]);
  const dtw = new DynamicTimeWarping(
    detectedChromaHistory,      // 12 chroma values
    expectedChromaHistory,      // 12 chroma values
    cosineDistance              // Method of comparing chroma vectors
  );
  const distance = dtw.getDistance();
  const normalisation = distance / (detectedChromaHistory.length +
expectedChromaHistory.length);
  return {result: (normalisation) < 0.20, dist: distance, norm: normalisation};
}
```

Figure 12: Polyphonic chroma value comparison code

4.7 Detection Accuracy

The accuracy of the current implementation of polyphonic pitch detection has been represented through the use of iterative testing using the methods specified in Section 3.4. This data was gathered in a LibreOffice Calc worksheet to calculate detection accuracy and robustness (cf. Appendix A). The values collected are the title of the distance function implemented, the measure tested, the calculated distance after DTW processing, the normalisation of the acquired distance, the threshold used against the normalisation to determine whether the note was detected, the number of consecutive matches required to count as a match, the outcome after each attempt of playing a note, and additional comments for each attempt. This contains the successful pitch detections, the number of undetected notes and false positives that occurred. The file used for testing is a manually created MusicXML file that consists of four measures for testing different abilities of the system's pitch detection (cf. Figure 13).



Figure 13: MusicXML score created for testing pitch detection

The method used for testing was iterating through the distance functions while varying the threshold between 0.2 and 0.5 as this was found during development to be the range in which detections are successful and where false positives begin to appear as the threshold increases. For the testing, an M-Audio M-Track Solo audio interface was used with a Yamaha YDP-143 eighty-eight-key piano and demonstrated

that the sum of squared, Euclidean, and Chebyshev distance formulas were able to detect pitches, but only in higher octaves where the overtones are less invasive to the signal. This is unusable and unreliable when playing against sheet music. The Manhattan, Minkowski, and Canberra formulas were not able to detect any pitches within the threshold range when playing on all the octaves of a piano. These are not viable for pitch detection as notes being detected are a fundamental requirement for this system. The only methods that have shown a glimpse of robustness and reliability are the cosine and Pearson methods as they were able to detect notes in the expected octave and were able to detect polyphonic pitches.

The Pearson formula was less reliable than the cosine formula as a C chord could not be detected as notated in the score but was capable when the C4 note of the chord was replaced with a C5. But to detect the correct note, it consistently took multiple attempts of detection with only minimal instances of false positives. The cosine formula resulted in the most reliable formula for pitch detection for this system as when the threshold is between 0.22 and 0.23, the number of attempts required for detection is lower than the Pearson formula, the expected notes did not have to be adjusted to be detected, and the overall robustness was more consistent than the other formulas.

4.8 Dilemmas

Throughout my development, several issues appeared which has hindered the development of this system. Ideal images of scores and correctly formatted MIDI files are sufficient for conversion when using OMR, however the slight imperfections in score images, as well as unexpected problems within the software cause errors to occur when automatically converting files on the server (cf. Figure 14). This issue is beyond my understanding of OMR and unable to be resolved within the deadline of this system's creation. Once further development is achieved in either Audiveris' GitHub or other methods of achieving OMR, then the system may function with improved success.

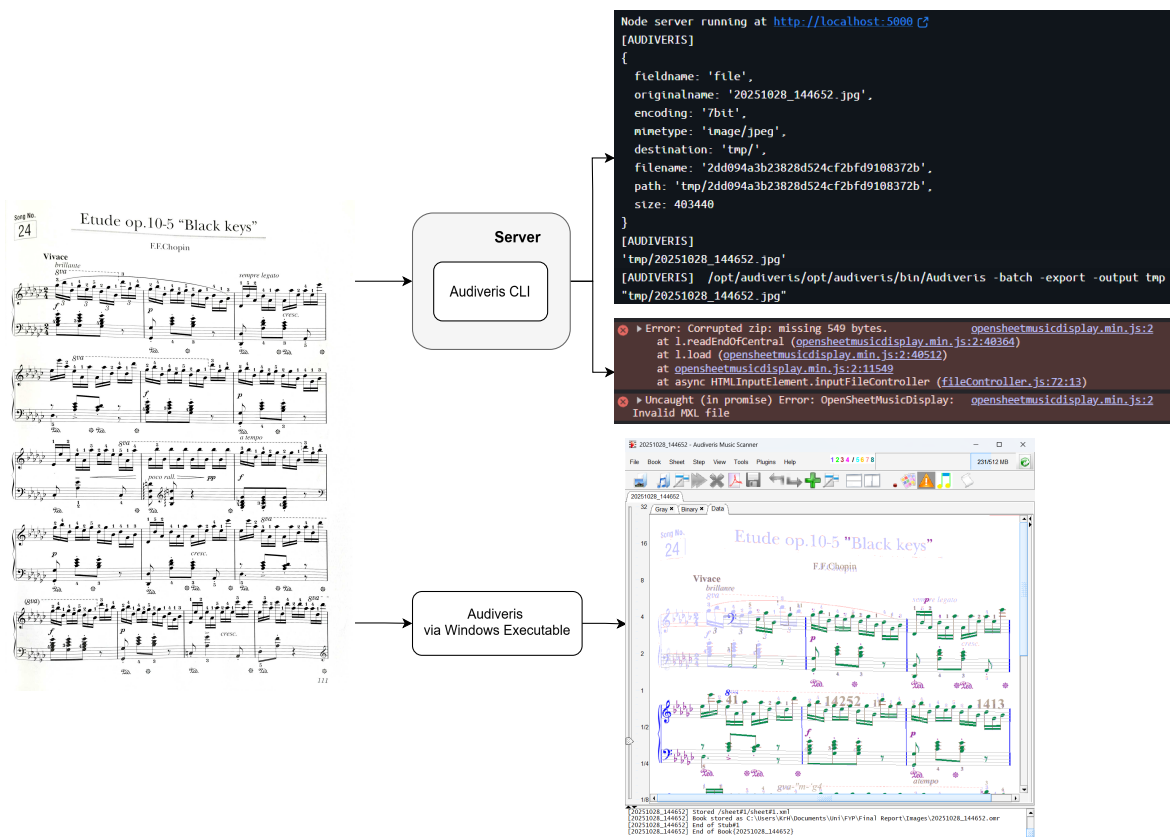


Figure 14: Errors resulting from malformed score conversion

The OSMD cursor functions by getting notes individually as the song is played, this causes issues for polyphonic detection as gathering an array of notes from the music score as it renders does not include repeat sections as there is no automatic detection for those tags in the score format which causes the expected notes to not line up with the notes detected by the current location of the cursor. This prevents the creation of a chromagram of the score which is a required step in many papers (Fremerey et al., 2009) as it requires a chromagram of both the score and audio.

Additionally, the method proposed by Fremerey et al. (2009) that compares the chroma value representations ignores the key in which a song is being played. This causes various issues and can be seen when playing an octave, since both notes of this harmony are expected to be played but a singular note can satisfy the match condition as the expected and detected chroma values register the same pitch class but not number of notes. This signifies that pitch class alone cannot be solely used to create polyphonic pitch detection, but the key of the notes must also be accounted for. This is also seen when a note has to be played twice in a row but the lingering harmonics of the audio can cause the cursor to skip the note entirely, a factor that may be resolved with the input of tempo within the system. Tied notes are problematic as both the beginning and end of the note must be played for detection, even though only the first instance should be played and the subsequent one ignored.

The score chroma values display the perfect expected note as it does not account for the noise that is included in the audio. This disparity between the two chroma values causes the normalised distance to be greater than expected, where 0 would indicate a perfect match while the average for the cosine function of 0.18 that the testing provides (cf. Section 4.7) signifies a similar match.

5 Conclusions

5.1 Introduction

This chapter will reflect on the unexpected challenges that have appeared during my research and development of the creation of a web application to provide real-time feedback to musicians based on uploaded music sheets and live audio input.

5.2 Challenges

The challenges that have arisen during the development of this system revolve around time, factors that have not appeared during research, and unexpected issues during development. This is highlighted in the score conversion since, unless the desired music sheet is manually edited and converted, the success rate for images into MusicXML is not to a satisfactory rate, as well as the problems that have arisen from the comparison of chroma values for polyphonic pitch detection. These issues, as outlined in Section 4.8, are possible to be resolved through alternative methods and implementations.

5.3 Development

The system's development progressed smoothly until the point of polyphonic pitch detection. Monophonic pitch detection was achievable when acquiring singular detected pitches and comparing it to the expected pitch of within a music score, as seen in a branch labelled "Monophonic-Pitch-Detection" and via a button labelled "Detection Method" on the branch labelled "Main" on GitHub (Hellstrom, K., 2026a). The challenge of advancing from monophony to polyphony is achievable when using open source technologies with further understanding of the libraries used during development, exploration into additional research papers concerning polyphonic detection, and the implementation of additional features that can assist in learning music.

5.4 Summary

This project focused on the creation and challenges of developing a web application which implements the functionality of existing lesson apps with OMR conversion to allow musicians to gain useful and reliable feedback on their own uploaded music sheets during practice. This application was tested using publicly available data along with my own personal data and instruments and has managed to achieve monophonic detection with attempts that have focused on polyphonic pitch detection based on research concluded on audio signal processing.

References

- Audiveris (2026) *Audiveris: Open-source optical music recognition* (Version 5.9.0) [Online].
Available at: <https://github.com/Audiveris/audiveris> (Accessed: 30 Oct. 2025)
- Carabias-Orti, J.J. et al. (2015) *An audio to score alignment framework using spectral factorization and dynamic time warping* [Online].
Available at: <https://archives.ismir.net/ismir2015/paper/000094.pdf> (Accessed: 1 Oct. 2025)
- Cheriell (2021) *Aligned classical piano audio and score (ACPAS) dataset* [Online].
Available at: <https://github.com/cheriell/ACPAS-dataset> (Accessed: 22 Dec. 2025)
- Chordify BV (2026) *Chordify: Learn and play your favorite songs* [Online].
Available at: <https://chordify.net> (Accessed: 24 Oct. 2025)
- dynamic-time-warping (2016) *dynamic-time-warping* [Online].
Available at: <https://www.npmjs.com/package/dynamic-time-warping> (Accessed: 21 Dec. 2025)
- Flowkey GmbH (2026) *Flowkey: Learn piano* [Online].
Available at: <https://www.flowkey.com/en> (Accessed: 24 Oct. 2025)
- Fremerey, C., Muller, M. and Clausen, M. (2009) *Towards Bridging the Gap between Sheet Music and Audio* [Online], pp. 1-11.
Available at: <https://drops.dagstuhl.de/storage/16dagstuhl-seminar-proceedings/dsp-vol09051/DagSemProc.09051.8/DagSemProc.09051.8.pdf> (Accessed: 2 Oct. 2025)
- Greenjava (2014) *OpenOMR* [Online].
Available at: <https://github.com/greenjava/OpenOMR> (Accessed: 20 Dec. 2025)
- Hawthorne, C. et al. (2018) *Onsets and Frames: Dual-Objective Piano Transcription* [Online], arXiv, p. 6
Available at: <https://doi.org/10.48550/arXiv.1710.11153> (Accessed: 28 Oct. 2025)
- International Music Score Library Project (2026). *IMSLP/ Petrucci Music Library*. [Online].
Available at: <https://imslp.org/> (Accessed: 10 Nov 2025).
- Kurth, F. et al. (2007) *Automated Synchronization of Scanned Sheet Music with Audio Recordings* [Online], pp. 2-6
Available at: https://ismir2007.ismir.net/proceedings/ISMIR2007_p261_kurth.pdf (Accessed: 2 Oct. 2025)
- Kyle Hellstrom (2026a) *Real-time Interactive Music Score Tracking Using Live Microphone Input* [Online].
Available at: <https://github.com/Kyle-R-H/FYP/> (Accessed: 9 Nov. 2026)
- Kyle Hellstrom (2026b) *Publicly available scores* [Online].
Available at: <https://github.com/Kyle-R-H/FYP/tree/Monophonic-Pitch-Detection/input> (Accessed: 9 Nov. 2026)
- LibreScore (2023) *webmscore* [Online].
Available at: <https://www.npmjs.com/package/webmscore> (Accessed: 16 Feb. 2026)
- Makemusic (2022) *Music in MusicXML: A list of sites with sheet music in musicxml format* [Online].
Available at: <https://www.musicxml.com/music-in-musicxml/> (Accessed: 28 Oct. 2025)
- Manhtai (2024) *Public domain MusicXML files* [Online].
Available at: <https://github.com/musetrainer/library/tree/master> (Accessed: 18 Mar. 2026)
- Math.Net Numerics (2024) *Distance Metrics* [Online].
Available at: <https://numerics.mathdotnet.com/Distance> (Accessed: 18 Mar. 2026)

Melobytes (2026) *MIDI to MusicXML converter* [Online].
Available at: <https://melobytes.com/en/app/midi2musicxml> (Accessed: 16 Feb. 2026).

Meyda (2026) *Meyda: Audio feature extractor* [Online].
Available at: <https://github.com/meyda/meyda> (Accessed: 24 Oct. 2025)

Muse Group (2025) *Musescore: free music notation software* [Online].
Available at: <https://musescore.org/en> (Accessed: 24 Oct. 2025)

Music21 (2025) *Music21* [Online].
Available at: <https://pypi.org/project/music21/> (Accessed: 16 Feb. 2026)

Noteflight (n.d.) *Noteflight online music notation software* [Online].
Available at: <https://www.noteflight.com/> (Accessed: 16 Feb. 2026).

OpenSheetMusicDisplay (2026) *A musicxml renderer for the browser* [Online].
Available at: <https://github.com/opensheetmusicdisplay/opensheetmusicdisplay> (Accessed: 24 Oct. 2025)

Organum Ltd (2026a) *PlayScore 2* [Online].
Available at: <https://www.playscore.co/> (Accessed: 24 Oct. 2025)

Organum Ltd (2026b) *What is optical music recognition* [Online].
Available at: <https://www.playscore.co/blog/what-is-optical-music-recognition> (Accessed: 24 Oct. 2025)

Pan, X. et al. (2015) *Musical examination to bridge audio data and sheet music* [Online], pp. 3-5.
Available at: <https://doi.org/10.1117/12.2083407> (Accessed: 2 Oct. 2025)

Ritwick Dey (2026) *Live Server* [Online].
Available at: <https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer> (Accessed: 26 Jan. 2026)

Scanscore Ltd (2026a) *ScanScore* [Online].
Available at: <https://scan-score.com/en/> (Accessed: 11 Dec. 2025)

Scanscore Ltd (2026b) *What is optical music recognition software* [Online].
Available at: <https://scan-score.com/en/what-is-optical-music-recognition-software> (Accessed: 11 Dec. 2025)

Sengpiel, E. (2014) *Key frequencies of a piano* [Image], Sengpiel Audio.
Available at: <https://sengpielaudio.com/calculator-notenames.htm> (Accessed: 19 Dec. 2025)

Simply Ltd (2026) *Simply Piano: Learn piano fast* [Online].
Available at: <https://www.hellosimply.com/simply-piano> (Accessed: 24 Oct. 2025)

Synthesia LLC (2026) *Synthesia* [Online].
Available at: <https://synthesiagame.com> (Accessed: 24 Oct. 2025)

TadaoYamaoka (2025) *Vocal Pitch Monitor* (Version 1.5.5) [Online].
Available at: <https://play.google.com/store/apps/details?id=com.tadaoyamaoka.vocalpitchmonitor> (Accessed: 1 Mar. 2026)

TrueFire Inc (2026) *TrueFire: Online guitar lessons* [Online].
Available at: <https://truefire.com> (Accessed: 11 Dec. 2025)

Ultimate Guitar (2026) *Ultimate Guitar Tabs* [Online].
Available at: <https://www.ultimate-guitar.com> (Accessed: 24 Oct. 2025)

Wilson, C. (2015) *PitchDetect: Pitch detection in Web Audio using autocorrelation* [Online].
Available at: <https://github.com/cwilso/PitchDetect/blob/main/js/pitchdetect.js> (Accessed: 5 Mar. 2026)

World Wide Web Consortium (W3C) (2024) *Web Audio API 1.1* [Online]
Available at: <https://www.w3.org/TR/webaudio/> (Accessed: 20 Dec. 2025)

Yousician Ltd (2026) *Yousician: Learn & play guitar* [Online].
Available at: <https://yousician.com> (Accessed: 24 Oct. 2025)

Appendix A - Pitch Detection Testing Dataset

This appendix presents a sample of the dataset used to evaluate the DTW pitch detection system. The complete dataset is available in the project GitHub:

https://github.com/Kyle-R-H/FYP/blob/0af3bb80804ce83beb3f9c997c7669b9864e873f/testing/pitch_detection_testing_dataset.ods

Function	Measure	DTW Distance	Distance Norm	Threshold	Consecutive Matches	Outcome	Comments
SSD	1	4.763	0.149	0.2	3	Match	+2 octave
SSD	1	4.173	0.13	0.2	3	Match	+2 octave
SSD	1	5.073	0.159	0.2	3	Match	+2 octave
SSD	1	5.148	0.161	0.2	3	Match	+2 octave
SSD	1	4.836	0.151	0.2	3	Match	+2 octave
SSD	2			0.2	0	Undetected	Undetectable
SSD	1	8.647	0.27	0.3	3	Match	+2 octave
SSD	2	7.081	0.221	0.3	3	Match	+2.5 octave (G6 + C7) Unrealistic
SSD	1	8.647	0.27	0.4	3	Match	+2 octave
SSD	2	7.081	0.221	0.4	3	Match	+3 octave Unrealistic
Further Testing is futile, higher and lower thresholds acted the same as the previous results, all later measures became undetectable							
Euclidean	1			0.2	0	Undetected	Undetectable
Euclidean	1	9.071	0.283	0.3	3	Match	+3 octave
Euclidean	2			0.3	0	Undetected	Undetectable
Euclidean	1	9.464	0.296	0.4	3	Match	+2 octave
Euclidean	2	11.668	0.365	0.4	3	Match	+3 octave Unrealistic
Euclidean	1	15.767	0.493	0.5	3	Match	+1 Octave
Euclidean	2	15.62	0.488	0.5	3	Match	+1 Octave
Euclidean	3	15.239	0.476	0.5	3	Match	+2 octave
Euclidean	4	15.824	0.495	0.5	3	Match	+3 octave
At threshold 0.5, the detection becomes unreliable as notes start to sustain and cause false positives.							
Manhattan					0	Undetected	
No matter the threshold, notes are not detectable							
Chebyshev	1	8.481	0.265	0.3	3	Match	+2 octave
Chebyshev	2	8.11	0.253	0.3	3	False Positive	Consistently immediate detection after measure 1
Chebyshev	2	9.16	0.286	0.3	3	Match	+1 octave
Chebyshev	3			0.3	0	Undetected	Undetectable
Unreliable detection after repeated trials							
Minkowski					0	Undetected	
No matter the threshold, notes are not detectable							
Canberra					0	Undetected	
No matter the threshold, notes are not detectable							

Function	Measure	DTW Distance	Distance Norm	Threshold	Consecutive Matches	Outcome	Comments
Cosine	1	4.703	0.147	0.15	3	Match	+1 octave
Cosine	2	4.274	0.134	0.15	3	Match	+1 octave, works with G3 and C4
Cosine	3	4.694	0.147	0.15	3	Match	+1 octave
Cosine	4	4.066	0.127	0.15	3	Match	+1 octave since pitch classes, it works with only C4
With threshold of 0.15, there are no false positives, just overtones causing problems							
Cosine	1	5.779	0.181	0.2	3	Match	+1 octave
Cosine	2	6.256	0.196	0.2	3	False Positive	Consistently immediate detection after measure 1
Cosine	1	5.692	0.178	0.2	3	Match	+1 octave
Cosine	2	6.282	0.196	0.2	3	False Positive	Consistently immediate detection after measure 1
Cosine	3	5.947	0.18	0.2	3	Match	+1 octave 1 st try and without noticeable delay
Cosine	4	5.23	0.163	0.2	3	Match	+1 octave 1 st try and without noticeable delay
Cosine	4	6.069	0.19	0.2	3	Match	Current octave 1 st try and without noticeable delay
Cosine	1	5.586	0.175	0.2	3	Match	+1 octave
Cosine	2	5.749	0.18	0.2	3	Match	+1 octave 1 st try and without noticeable delay
Cosine	3	6.116	0.191	0.2	3	False Positive	Consistently immediate detection after measure 2
With threshold of 0.2, single and double notes cause false positives in the +1 octave. Chord works in +1 octave as well as octave but only 1 note is required							
Cosine	1	7.033	0.22	0.22	3	Match	2 nd try, correct octave
Cosine	2	6.804	0.213	0.22	3	Match	2 nd try, correct octave
Cosine	3	6.785	0.212	0.22	3	Match	1 st try correct octave
Cosine	4	6.305	0.197	0.22	3	Match	Only C4 was required and was detected
Cosine	1	7.029	0.22	0.23	3	Match	1 st try correct octave
Cosine	2	7.236	0.226	0.23	3	False Positive	Consistently immediate detection after measure 1
Cosine	3	7.115	0.222	0.23	3	Match	1 st try correct octave
Cosine	4	6.537	0.204	0.23	3	Match	1 st try correct octave
Cosine	1	7.512	0.235	0.25	3	Match	1 st try correct octave
Cosine	2	7.229	0.226	0.25	3	False Positive	Consistently immediate detection after measure 1
Cosine	3	7.633	0.239	0.25	3	False Positive	Consistently immediate detection after measure 1
Cosine	4	7.15	0.223	0.25	3	False Positive	Consistently immediate detection after measure 1
The testing shows that the ideal threshold for the system and a piano is between 0.22 and 0.23 for robustness							
Pearson	1	5.629	0.176	0.2	3	Match	3 rd try, correct octave
Pearson	2	5.7	0.178	0.2	3	Match	1 st try correct octave
Pearson	3	5.203	0.163	0.2	3	Match	Chord worked 1 st try, 1 st inversion
Pearson	4	5.188	0.162	0.2	3	Match	1 st try correct octave, but sometimes is a false positive from the chord
Pearson	1	5.435	0.17	0.2	3	Match	1 st try correct octave
Pearson	2	5.61	0.175	0.2	3	Match	1 st try correct octave
Pearson	3	6.143	0.192	0.2	3	Match	Chord worked 1 st try, 1 st inversion
Pearson	4	5.778	0.181	0.2	3	False Positive	Immediate detection after measure 3
Pearson	1	5.475	0.171	0.2	3	Match	4 th try, correct octave
Pearson	2	5.947	0.186	0.2	3	Match	3 rd try, correct octave
Pearson	3	6.357	0.199	0.2	3	Match	Chord worked 1 st try, 1 st inversion
Pearson	4	4.994	0.156	0.2	3	Match	1 st try, correct octave
Pearson	1	5.399	0.169	0.19	3	Match	4 th try, correct octave
Pearson	2	5.684	0.178	0.19	3	Match	2 nd try, correct octave
Pearson	3	5.89	0.184	0.19	3	Match	Chord worked 1 st try, 1 st inversion
Pearson	4	4.923	0.154	0.19	3	Match	1 st try, correct octave
Pearson	1	5.462	0.171	0.21	3	Match	2 nd try, correct octave
Pearson	2	5.735	0.179	0.21	3	Match	2 nd try, correct octave
Pearson	3	6.487	0.203	0.21	3	Match	Chord worked 1 st try, 1 st inversion
Pearson	4	5.796	0.181	0.21	3	Match	2 nd try, correct octave
Unreliable, often only works 2 nd or 3 rd attempt and cannot play root inversion chords							